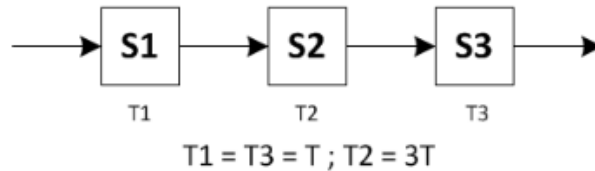


Práctico 4 - AdC

Procesador con Pipeline

Ejercicio 1:

En un microprocesador con tres etapas de pipeline: $S_1 \rightarrow S_2 \rightarrow S_3$, con tiempos de ejecución $T_1 = T_3 = T$ y $T_2 = 3T$.



- a) ¿Cuál de los tres segmentos o etapas causa la congestión? (el cuello de botella). *S2*
- b) Asumiendo que el segmento problemático se puede dividir en dos etapas consecutivas, ninguna de ellas con duración menor que T , ¿cuál sería la mejor partición posible? ¿Cuál sería el período de *clock* resultante para este nuevo pipeline de 4 etapas?
- c) Asumiendo que el segmento problemático se puede dividir en varias etapas consecutivas de duración T , ¿cuál es el período de *clock* del microprocesador? ¿Cuál es el tiempo de ejecución entre instrucciones?
- b) Subdivido S_2 en dos de tal manera que cada parte tenga $1.5T$ de tiempo de ejecución. Ahora el período de clock sería de $1.5T$
- c) El período de clock será de T , al igual que el tiempo de ejecución entre instrucciones.

Ejercicio 2:

Asumiendo que las etapas de un procesador ARM tienen las siguientes latencias:

IF	ID	EX	MEM	WB
30ns	10ns	9ns	40ns	20ns

- a) ¿Cuánto tiempo se requiere en un microprocesador sin pipeline para completar la ejecución de la instrucción de mayor latencia, es decir, la latencia del procesador completo?
- b) Si se requiere ejecutar esa instrucción en un microprocesador con pipeline, ¿a qué velocidad debería trabajar el *clock*?
- c) ¿Cuál es el tiempo de ejecución de una instrucción en un microprocesador con pipeline? ¿Cada cuanto se ejecuta una nueva instrucción en este procesador?
- d) Si un microprocesador con pipeline ejecuta 3 instrucciones consecutivas ¿cuál es la ganancia de velocidad de un procesador con pipeline respecto de uno sin pipeline? ¿Y si se ejecutan 1000 instrucciones consecutivas?

$$\text{Ganancia de velocidad} = \frac{\text{Tiempo de ejecución sin pipeline}}{\text{Tiempo de ejecución con pipeline}}$$

a) Para completar la ejecución de una instrucción que tiene la latencia del procesador completo se requerirá la sumatoria de todas las latencias de cada una de las etapas del micro: $30 + 10 + 9 + 40 + 20 = 109\text{ns}$

b) El clock debería trabajar a 40ns para ejecutar dicha instrucción en un micro con pipeline, que es la mayor latencia de entre todas las partes del micro.

c) El tiempo de ejecución de una instrucción en un micro con pipeline será de 200ns , ya que a todas las instrucciones les tomará 5 ciclos de clock ejecutarse. ($40 * 5 = 200$). Una nueva instrucción se ejecuta cada 40ns .

d)

Ejecutar 3 instrucciones s/ pipeline: $109\text{ns} * 3 = 327\text{ns}$

Ejecutar 3 instrucciones c/ pipeline:

200ns (1° instrucción) + $2 * 40\text{ns}$ (siguientes dos instrucciones) = 280ns

Ganancia: $327\text{ns} / 280\text{ns} = 1.16$

Ejecutar 1000 instrucciones s/ pipeline: $109\text{ns} * 1000 = 109\text{ms}$

Ejecutar 1000 instrucciones c/ pipeline: $200\text{ns} + 999 * 40\text{ns} = 40.16\text{ms}$

Ganancia: $109 / 40.16 = 2.71$

Ejercicio 3:

Asumiendo que las etapas individuales del pipeline de dos procesadores ARM distintos tienen las siguientes latencias:

Caso	IF	ID	EX	MEM	WB	Unidad
1	300	400	350	500	100	ps
2	200	150	120	190	140	ps

a) ¿Cuál es el ciclo de *clock* para la versión con y sin pipeline para cada caso?

b) ¿Cuál es la latencia de la instrucción **LDUR** para ambos, considerando las versiones de procesador con y sin pipeline?

c) Si se pudiera partir una etapa del pipeline en dos nuevas etapas, cada una con la mitad de la latencia de la etapa original, ¿Que etapa elegiría y cuál será el nuevo ciclo de *clock*?

a) Caso 1

clock s/ pipeline: 1650ps

clock c/ pipeline: 500ps

Caso 2

clock s/ pipeline: 800ps

clock c/ pipeline: 200ps

b) LDUR pasa por las etapas IF, ID, EXC, MEM y WB.

Caso 1:

micro s/ pipeline: 1650ps (máxima latencia para una instrucción que pasa por todas las etapas)

micro c/ pipeline: $500 * 5 = 2500\text{ps} = 2.5\text{ns}$

Caso 2:

micro s/ pipeline: 800ps

micro c/ pipeline: $200 * 5 = 1000\text{ps} = 1\text{ns}$

c) Caso 1: parto MEM en 250ps y 250ps -> nuevo ciclo de clock: 400ps

Caso 2: parto IF en 100ps y 100ps -> nuevo ciclo de clock: 190ps

Ejercicio 4:

Dados los siguientes fragmentos de código de instrucciones LEGv8:

A	B	C
<pre> 1> SUB X6, X1, X3 2> SUB X7, X6, X5 </pre>	<pre> 1> ADDI X10, X1, #8 2> LDUR X3, [X10, #0] 3> SUB X3, X2, X10 </pre>	<pre> 1> LDUR X1, [X10, #0] 2> LDUR X2, [X10, #8] 3> ADD X3, X1, X2 4> STUR X3, [X10, #24] 5> LDUR X4, [X10, #16] 6> ADD X5, X1, X4 7> STUR X5, [X10, #32] </pre>

a) Analizar en el código las dependencias de datos. En cada caso indicar: los números de las instrucciones involucradas y el operando en conflicto.

b) Mostrar el orden de ejecución de las instrucciones y determinar cuales generan *data hazards*. En cada caso indicar en qué etapa se genera el hazard.

b) Mostrar el orden de ejecución de las instrucciones utilizando un procesador con *forwarding stall*.

c) Reescribir la sección de código alterando el orden de las instrucciones para evitar *stalls* innecesarios. Mostrar el nuevo orden de ejecución.

d) ¿Cuántos ciclos toma la ejecución del código en cada caso?

A:

Dependencia	Instancia	Registro	Hazard?
RAW	1-2	X6	Sí

Ejecución con forwarding stall

	C1	C2	C3	C4	C5	C6
1. Sub	IF	ID	EXC	Mem	Wb	
2. Sub		IF	ID	EXC	Mem	Wb

Tomó 6 ciclos de ejecución

B:

Dependencia	Instancia	Registro	Hazard?
RAW	1-2	X10	Sí
RAW	1-3	X10	Sí

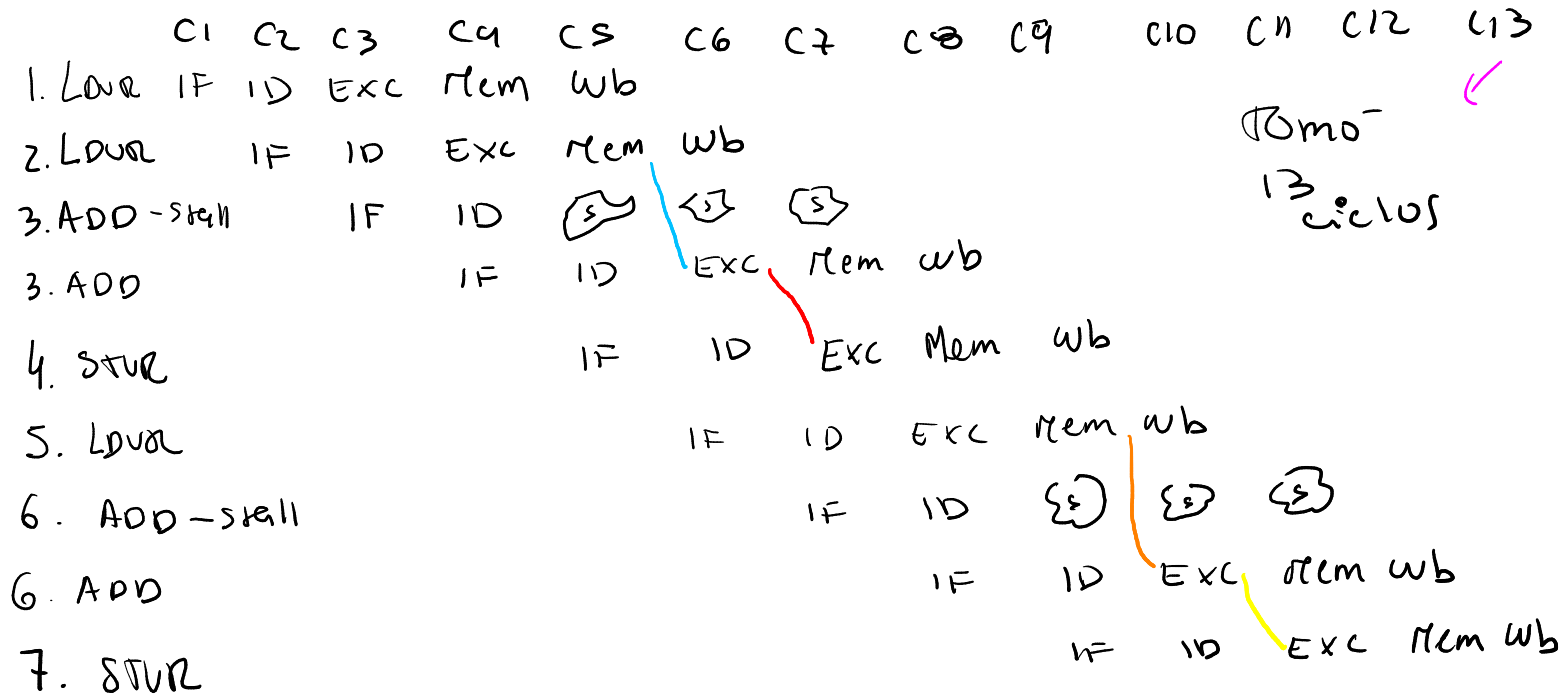
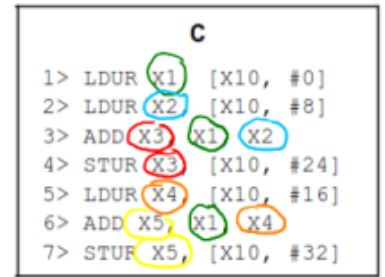
Ejecución con forwarding stall

	C1	C2	C3	C4	C5	C6	C7
1. ADDI	IF	ID	EXC	Mem	Wb		
2. LDUR		IF	ID	EXC	Mem	Wb	
3. SUB			IF	ID	EXC	Mem	Wb

Tomó 7 ciclos de clock

C:

Dependencia	Instancia	Registro	Hazard?
Raw	1-3	X ₁	Si
Raw	1-6	X ₁	NO
Raw	2-3	X ₂	Si
Raw	3-4	X ₃	Si
Raw	5-6	X ₄	Si
Raw	6-7	X ₅	Si



Para evitar stalls reordeno:

LDUR X₁, [X10, #0]

LDUR X₂, [X10, #8]

LDUR X₄, [X10, #16]

ADD X₃, X₁, X₂

STUR X₃, [X10, #24]

ADD X₅, X₁, X₄

STUR X₅, [X10, #32]

=> Se ejecuta un ciclo más tarde, no necesito el stall

=> el LDUR del que dependia se ejecuta mucho más antes.

Ejercicio 6:

Dado el siguiente fragmento de código de instrucciones LEV8:

```

1>      SUB X3, X1, X2
2>      CBZ X3, skip
3>      ADD X4, X3, XZR
4>      ORR X7, X1, X2
5>      AND X1, X6, X2
6>      STUR X5, [X2, #40]
7> skip: ADD X5, X3, XZR
  
```

a) Analizar en el código las dependencias de datos y de control, determinar cuales generan hazards. En cada caso indicar: los números de las instrucciones involucradas, en qué etapa se encuentra c/u, el operando en conflicto y el tipo de hazard.

b) Mostrar el orden de ejecución de las instrucciones utilizando un procesador con forwarding stall suponiendo que $X1 \neq X2$. ¿Cuál es la penalidad por el hazard de control?

c) Mostrar el orden de ejecución de las instrucciones utilizando un procesador con forwarding stall suponiendo que $X1 = X2$. ¿Cuál es la penalidad por el hazard de control?

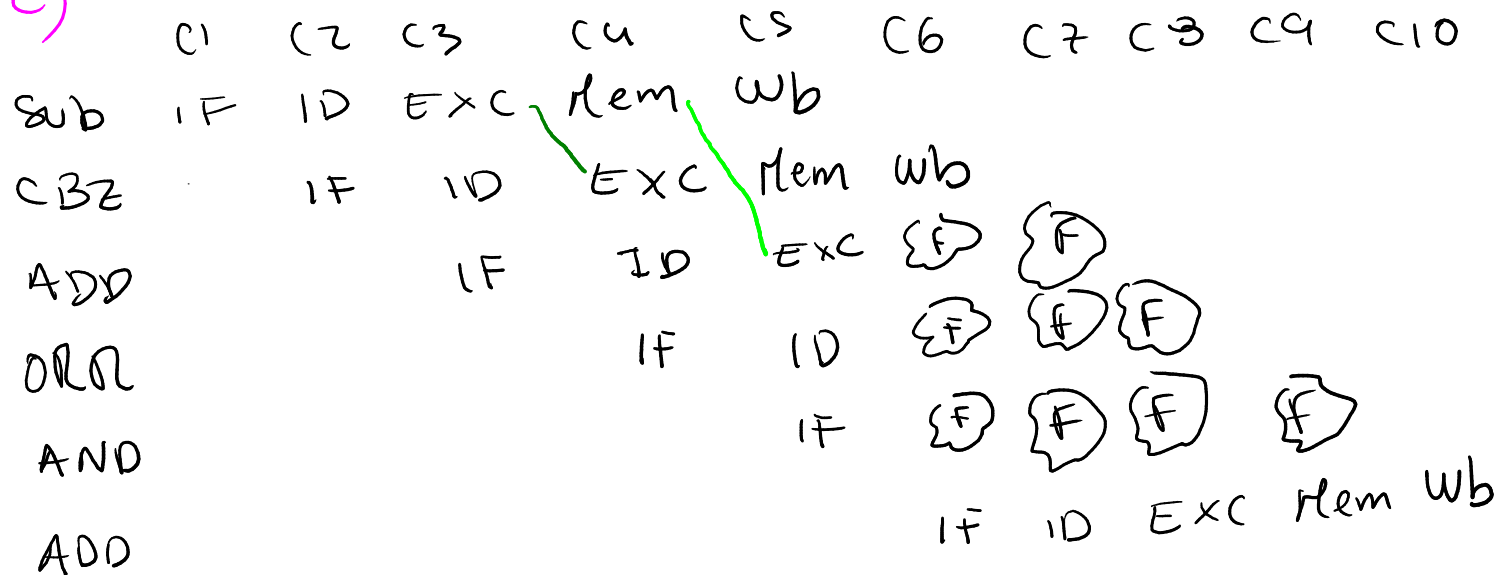
Dependencia	Instancia	Registro	Hazard?
RAW	1-2	X3	Si
RAW-Cond.	1-7	X3	NO
Control	2	-	Si Salta
RAW-Cond	1-3	X3	Si

b)

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
SUB	IF	ID	EXC	Mem	wb						
CBZ		IF	ID	EXC	mem	wb					
ADD			IF	ID	EXC	mem	wb				
ORR				IF	ID	EXC	mem	wb			
AND					IF	ID	EXC	mem	wb		
STUR						IF	ID	EXC	mem	wb	
ADD							IF	ID	EXC	mem	wb

No hay penalización.

c)



La penalización es de 3 ciclos.

Sin flush, hubiera terminado en el ciclo 7.

Ejercicio 7:

Para el siguiente fragmento de código de instrucciones LEGv8 que se ejecuta en un procesador con forwarding stall:

```

1> SUB X3, X1, X2
2> ORR X8, X5, X6
3> L: LDUR X0, [X3, #0]
4> ADD X3, X3, X8
5> CBNZ X0, L
6> ADD X8, X0, X3
  
```

a) Analizar en el código las dependencias de datos y de control, determinar cuales generarían *hazards* y mediante qué técnica se evita. En cada caso indicar: los números de las instrucciones involucradas, el operando en conflicto y el tipo de hazard.

b) Mostrar el orden de ejecución y los recursos utilizados por las instrucciones para el segmento de código dado, suponiendo que CBNZ no se toma.

c) Mostrar el orden de ejecución y los recursos utilizados por las instrucciones para el segmento de código dado, suponiendo que CBNZ se toma una vez.

Dependencia	Instancia	Registro	Hazard?
Raw	1-3	X3	Si, fw
Raw	1-4	X3	NO
Raw	2-4	X8	Si, fw
Raw	3-5	X0	Si, fw
Raw-cond	3-6	X0	NO
Raw-cond	4-6	X3	Si, fw
Control	5	—	Si Salto (not-taken)
Raw-Cond	4-3	X3	Si
Raw-Cond	4-4	X3	NO

Dep.
si
no

Salto

Flush

Dep.

Si
Salto

